

UNIVERSITY OF
LIVERPOOL

COMP390

2025/26

Texas Hold’em

Qiaoyang Bi
201807716

BSc Computer Science

SCHOOL OF
COMPUTER SCIENCE & INFORMATICS

University of Liverpool
L69 3DR

Acknowledgements

I would like to express my gratitude to my supervisor, Wenjun Ye, for his technical guidance throughout the project.

I also thank my family and friends for their continuous support during the project.

Parts of this project's core logic and hand evaluation algorithm are based on the open-source work of Paz and Lilov on GitHub. I am also grateful to Kenney and Pixabay for providing the visual and audio assets. All third-party resources are used under their respective open-source or Creative Commons licenses.



COMP390

2025/26

Texas Hold’em

BSc Computer Science

SCHOOL OF
COMPUTER SCIENCE & INFORMATICS

University of Liverpool
L69 3DR

Abstract

This project is a multiplayer Texas Hold'em desktop game built with C#, WPF, and .NET 8.0. To handle network stability and prevent client-side cheating, the system implements an Authoritative Server architecture using Microsoft SignalR. The application supports up to 6 concurrent players online with low latency. Since the server processes all game rules and actions, players cannot manipulate the match results. The application also includes an offline mode for single-player practice.

In addition to standard Texas Hold'em rules, I implemented a custom "Card Swap" feature. This mechanic allows players to exchange one of their hole cards during a specific round to adjust their hand composition. For the core game engine, I wrote a hand-evaluation algorithm based on combinatorics (7 choose 5). This algorithm processes the card data to calculate hand ranks and determine the winner.

During the testing phase, I conducted functional tests on the core logic and network connections to verify system stability. Following this, I invited several users to test the application and provide feedback on the interface and gameplay. The final implementation integrates the evaluation algorithm and real-time networking framework to deliver a functional multiplayer poker system.

Statement of Ethical Compliance

Data Category: A
Participant Category: 2

I confirm that I have followed the ethical guidelines throughout this project. Further details can be found in the relevant sections of this dissertation.

Content

1 Introduction & Background 1

1.1 Context and Motivation 1

1.2 Problem Statement 1

1.3 Proposed Solution and Innovation 1

2. System Design 1

2.1 System Architecture 1

2.2 Algorithms and Data Structures 2

2.3 User Interface and Navigation 3

3. Implementation 8

3.1 Development Environment and Technologies 8

3.2 Data Persistence and User Authentication 9

3.2.1 Object-Relational Mapping and Database Design 9

3.2.2 RESTful API and Cryptographic Security 9

3.3 Core Game Logic Implementation 10

3.3.1 Hand Evaluation and Combinatorics Algorithm 10

3.3.2 Computer Logic 12

3.3.3 Original Card Swap Mechanism 13

3.4 Game State Machine and Action Handling 15

3.5 Real-time Network Communication 17

3.5.1 SignalR Configuration and WebSocket Protocol 17

3.5.2 Authoritative Server Implementation 18

4. Testing and Evaluation 19

4.1 Black-Box Functional Testing 19

4.2 User Acceptance Testing 22

4.2.1 Domain Terminology Inaccuracy: The "Call $0" Issue 22

4.2.2 Edge Case Resolution: Real-time State Deadlock in 1v1 Scenarios 23

4.2.3 Session Security and Concurrent Login Resolution 26

4.3 User Acceptance Testing Result Analysis 29

4.4 Summary Evaluation 30

5. Project Ethics 30

5.1 Ethical Compliance Categories 30
5.2 Informed Consent 30
5.3 Use of Dummy Data 30
5.4 Data Anonymity and Protection 30
6. Conclusion & Future Work 31
6.1 Conclusion 31
6.2 Future work 31
6.2.1 Advanced AI via Reinforcement Learning 31
6.2.2 Scalability via Redis Backplane 31
6.2.3 Competitive Social and Ranking Ecosystem 31
7. BCS Project Criteria and Self-Reflection 32
7.1 An ability to apply practical and analytical skills gained during the degree programme 32
7.2 Innovation and/or creativity 32
7.3 Synthesis of information, ideas and practices to provide a quality solution together with an evaluation of that solution 32
7.4 That your project meets a real need in a wider context 32
7.5 An ability to self-manage a significant piece of work 32
7.6 Critical self-evaluation of the process. 32
Reference 34
Appendix A: Installation Guide and User Manual (README) 35

1 Introduction & Background

1.1 Context and Motivation

Texas Hold'em is one of the most popular poker games in the world, where players combine two private cards with five shared community cards to form the best hand. My main motivation for this project comes from my strong interest in online game development. In fact, creating games was my primary reason for choosing a Computer Science major. Since I frequently play online poker with friends, I am very familiar with its rules and mechanics. During my research phase, I also studied and played HD Poker (HDpoker, 2026) to learn how professional web-based poker games handle user interfaces and game flow. In the past, I have only built single-player desktop applications. Therefore, for my final year project, I decided to challenge myself by developing a real-time multiplayer game, and also including the single player offline game mode.

1.2 Problem Statement

Building an online Texas Hold'em game is quite difficult and involves several major technical challenges. The first problem is the real-time network. I had never built a multiplayer game before this project. It is a challenge to ensure real-time game state synchronization across all connected clients. The second problem is how to design the computer players. The bots need to make good decisions, like choosing when to Call, Raise, Fold, or All-in. Because I added the new "Card Swap" rule, the computer also needs a special logic to decide if keeping a card or changing it is better. Finally, stopping players from cheating is a very important task. As with all online games, I must build a safe system to block any cheating behaviour.

1.3 Proposed Solution and Innovation

To solve the network and security issues, I built an Authoritative Server using Microsoft SignalR. This ensures that the server controls all the game rules, which stops players from cheating through their clients. For the project's creativity requirement, I added a new "Card Swap" feature. In normal Texas Hold'em, winning depends a lot on luck with the first two cards, and players cannot change what they receive. With my new feature, players get a chance to swap one card during a specific round. This gives players more control over their hands and makes the game require more strategy.

1. System Design

1.1 System Architecture

For the multiplayer features, the system uses a client-server model. To stop cheating and make the game fair, I used an Authoritative Server. This server handles all the main game logic, calculates the chips, and checks every player action. The client software only shows the game and takes user inputs. Therefore, any invalid actions or manipulated requests sent from the client are automatically rejected by the server.

I used Microsoft SignalR to handle the real-time networking in my game. The reason for the choice is that it enables low-latency two-way data transmission between the server and the client. This way, everyone in the game can see the same updates at the same time.

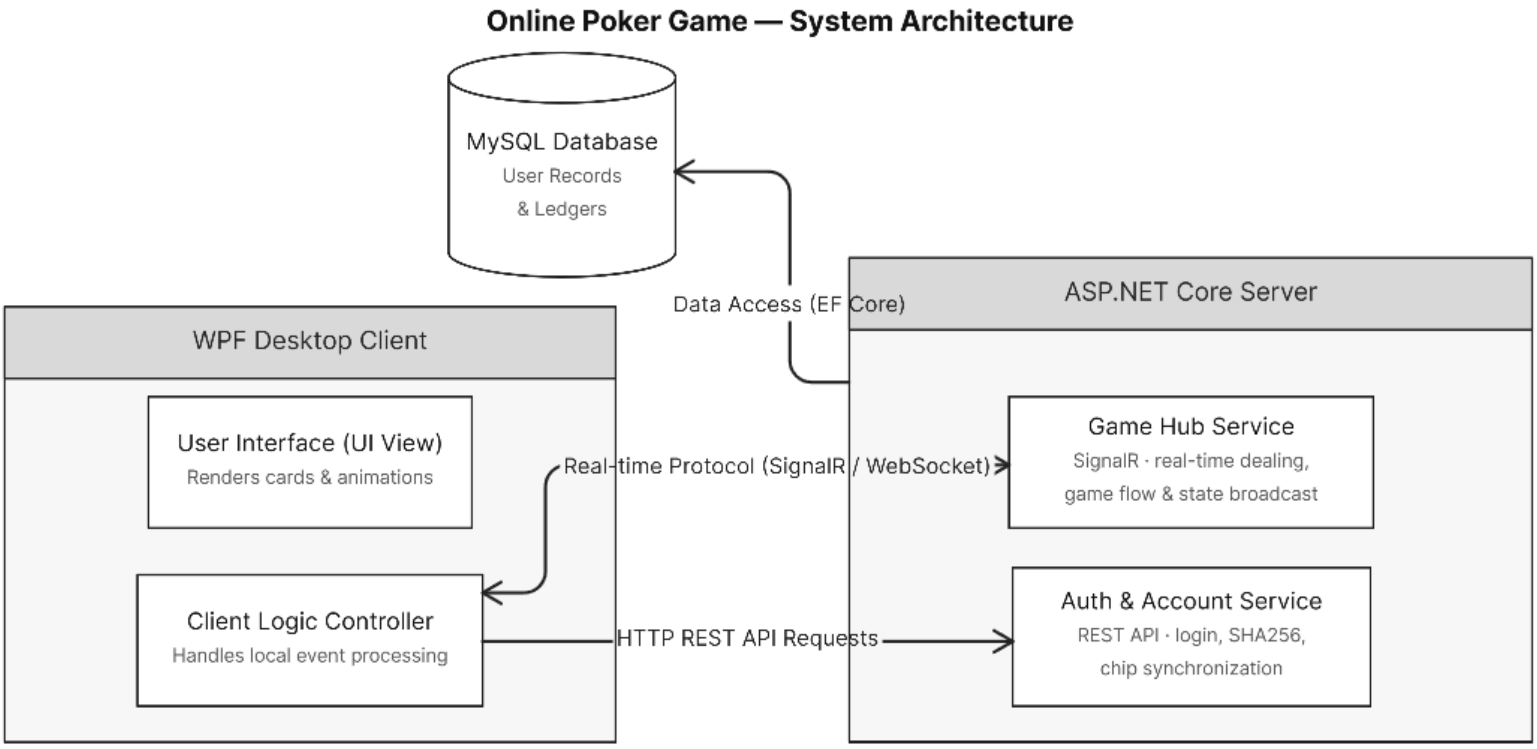


Figure 1: Client-Server Architecture using SignalR

1.2 Algorithms and Data Structures

I used Object-Oriented Programming to build this project. I created separate classes for each part of the poker game, such as the cards, the deck, and player profiles. This made the code much easier to manage and organise.

The hardest part of the system design is the hand evaluation algorithm. Based on Texas Hold'em rules, the game must find the best five cards out of seven. To do this, I used combinatorics. My algorithm creates all 21 possible five-card combinations from the seven cards. After that, it sorts the cards from highest to lowest and groups them to find hands like a flush or a full house. This sorting method is also very useful for tie-breakers because it compares the highest single cards left. With this logic, the system always finds the correct winner.

Figure 2 shows that the system's core logic has two main parts: Data Structures and the Hand Evaluation Algorithm.

For the data structures, the game uses a List<Card> to store the standard 52-card deck. I also used the Singleton pattern for the GameState so that the pot and the community cards remain the same across the whole application. Additionally, every player's data is stored in a PlayerState object, which tracks their current chips and bets in real-time.

For the Hand Evaluation Algorithm, the system needs to find the winner from a total of seven cards (two hand cards and five community cards). My code creates all 21 possible five-card combinations from these seven cards. Then, it checks the card patterns to find the hand ranks.

Finally, the CalculateHandScore() method changes each hand into a 64-bit integer. Because of this, the EvaluateShowdown() method can quickly compare the numbers and decide the final winner.

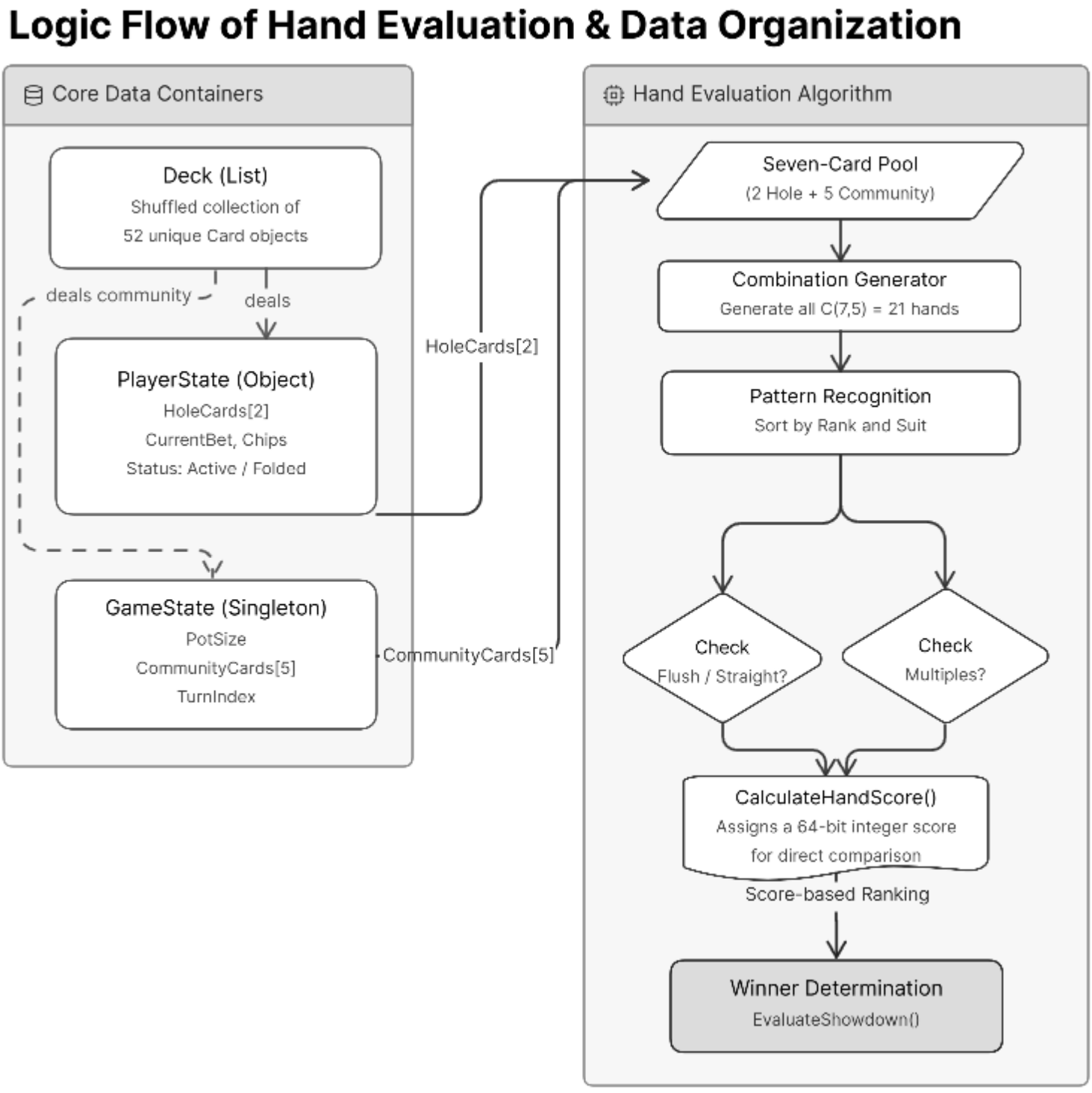


Figure 2: Logic Flow of Hand Evaluation and Core Data Structures

1.3 User Interface and Navigation

The game interface was implemented by Windows Presentation Foundation. The main menu serves as the entry point, providing navigation to the authentication interfaces. After logging

in, users can select between joining an online multiplayer room or initiating an offline practice session. Players can also entre the offline game without logging into an account.

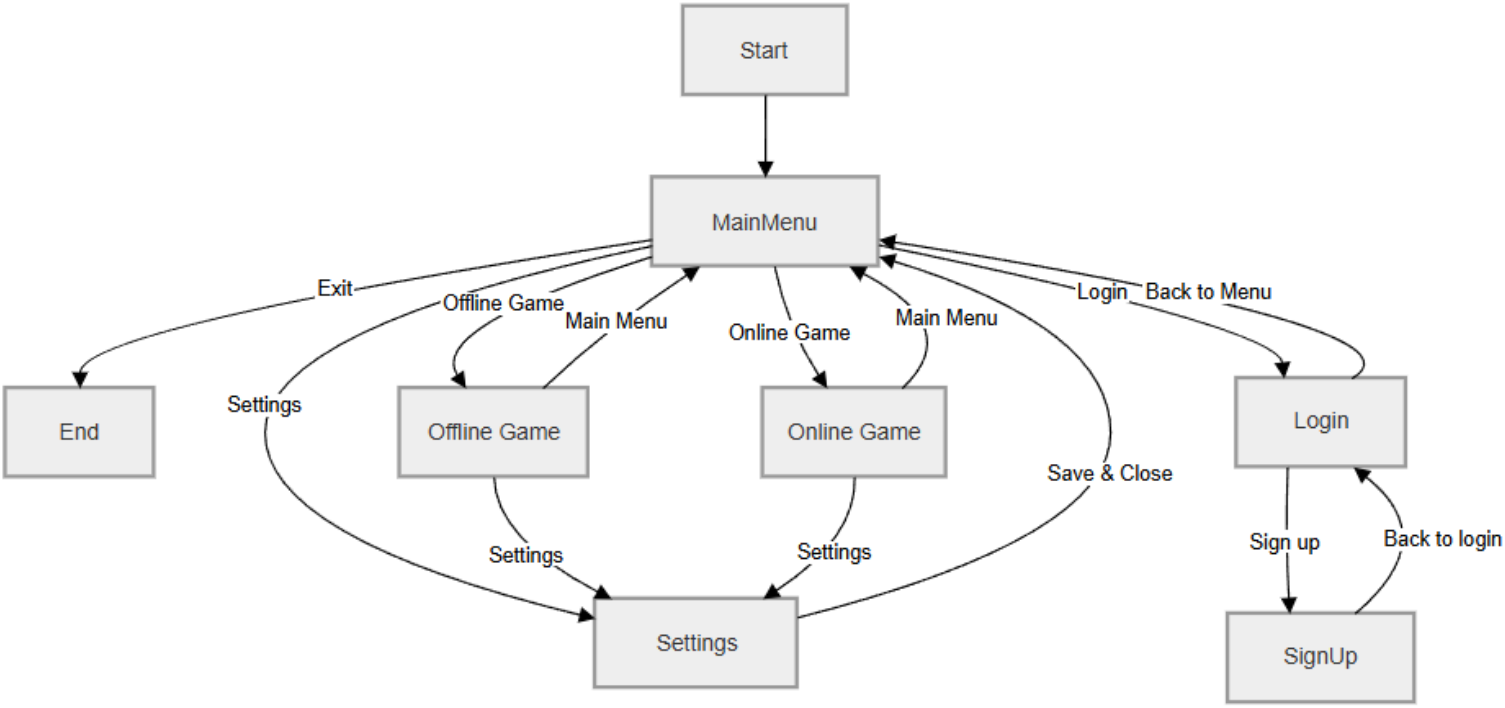


Figure 3: Sequence diagram of Texas Hold'em

To demonstrate the final visual outcome of the project, the following figures show screenshots of the application.

Main Menu and Authentication

Figure 4 shows the **main menu**, which serves as the application's entry point. The menu offers clear navigation buttons that allow users to access the login screen, start an offline training session, adjust game settings, or exit the program.

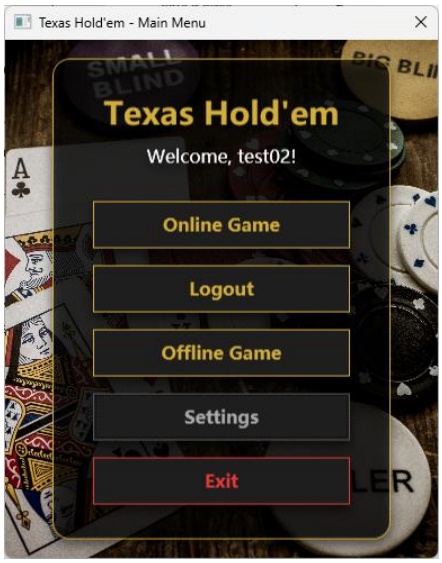


Figure 4: Main Menu interface of Texas Hold'em

The login interface requires users to enter their registered username and password to access online features. It includes an input form along with options to recover a forgot password or navigate to the registration page for new users.

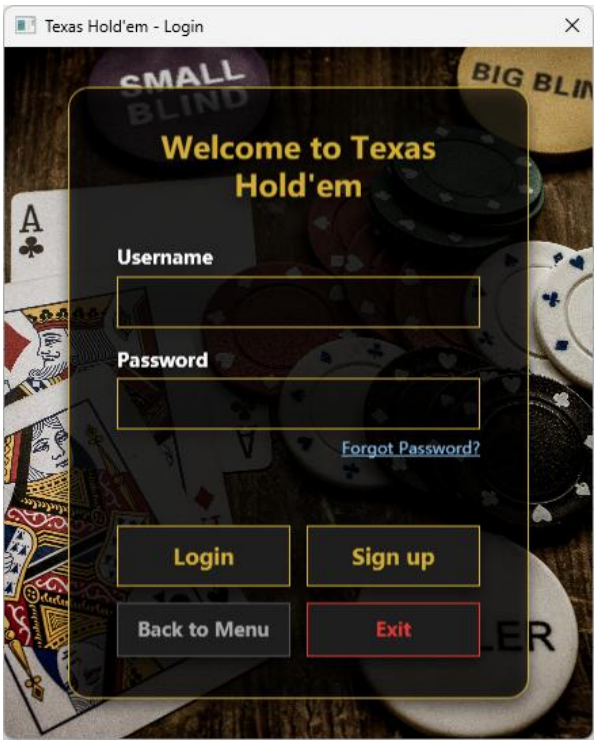


Figure 5: Login interface of Texas Hold’em

New players can use the **sign-up interface** to create an account. Users need to choose a unique username and a password. To make the account safer and help users if they forget their password later, the registration form also asks them to choose and answer two security questions, such as their favourite book or the name of their pet.

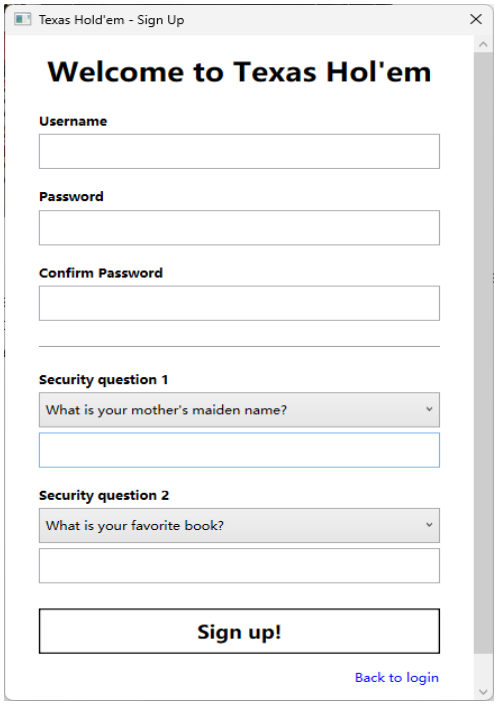


Figure 6: Registration interface of Texas Hold’em

Game Interfaces

The game interface is the most important part for players. The design uses a green poker table in the middle to show the community cards and the total pot. Each player's avatar, chips, and status are placed around the table. In the bottom-right corner, there are buttons for actions like 'Fold', 'Check', 'Raise', and 'All-in'. Also, there is a log panel in the bottom-left corner. It shows real-time information about the game and allows players to type messages to chat with others.



Figure 7: Online game interface of Texas Hold'em

The user interface changes based on the game state to help the player know what to do. During the game, if it is not the user's turn, the action buttons are disabled. This stops the user from clicking buttons and sending wrong requests to the server. When the server says it is the user's turn, the interface changes immediately. A gold border appears around the player's avatar, and a sound plays to notify them. At the same time, the betting buttons become active so the player can choose to bet or fold.

The offline training mode looks very similar to the online version, so the design stays consistent. However, the opponents on the screen are computer bots instead of real players. I added buttons to let users add or remove bots, which means they can change the difficulty level of the game. This mode is very useful for users because they can practice their poker skills and test the game rules without losing any virtual chips.



Figure 8: Offline game interface of Texas Hold'em

Customization

In the settings interface, it allows players to customize the game. The settings page is displayed in both English and Chinese, making it easier for players of different linguistic backgrounds to adjust the game settings. Players can use sliders to adjust the volume of background music and sound effects, change the desktop colour, and switch the interface language between English and Chinese. Also, there is a button for opening the game rules.

Figure 9: Settings interface of Texas Hold'em



2. Implementation

2.1 Development Environment and Technologies

To implement the system design, the primary integrated development environment (IDE) used for this project was Visual Studio 2022. This IDE provides native support for C# and the .NET 8.0 framework.

For the client application, I used Windows Presentation Foundation (WPF) to build the graphical user interface. To ensure I followed copyright rules, I used free music and images from Pixabay (2026) and Kenney (2026) for the game's frontend.

On the server side, I built the backend using ASP.NET Core and used Microsoft SignalR to handle the real-time multiplayer messages. For the database, I chose MySQL to store all the data. I used MySQL Workbench to design the tables and manage user information. The backend uses standard SQL queries to talk to the database. I also added Swagger UI to my

project, it can let me test the network requests and database connections easily when I haven't finish my frontend.

2.2 Data Persistence and User Authentication

To save the players' data and their account information, the system needs a reliable way to store data. Instead of writing manual SQL queries, Entity Framework Core was utilized on the server. This ORM tool streamlines the connection to the MySQL database and improves data management security.

2.2.1 Object-Relational Mapping and Database Design

I used Entity Framework Core's Code-First way to handle the database. Instead of writing long SQL scripts to make tables by hand, I just wrote C# classes. This approach ensures synchronization between the C# application models and the MySQL database schema. The most important part is the User model, which I used as the main plan to build the database tables.

Listing 1: The User entity model defining the database schema in User.cs

```
namespace TexasHoldemServer.Models
{
    public class User
    {
        public int UserID { get; set; }
        public string Username { get; set; } = string.Empty;
        public string PasswordHash { get; set; } = string.Empty;
        public string SecurityQ1 { get; set; } = string.Empty;
        public string SecurityA1 { get; set; } = string.Empty;
        public string SecurityQ2 { get; set; } = string.Empty;
        public string SecurityA2 { get; set; } = string.Empty;
        public int Chips { get; set; }
        public DateTime CreatedAt { get; set; }
    }
}
```

As demonstrated in Listing 1, this class encapsulates all important player attributes. The UserID serves as the primary key for relational indexing, while fields like SecurityQ1 and PasswordHash handle account security. By encapsulating this model within the AppDbContext's DbSet<User>, the server can utilise EF Core Migrations. Using EF Core makes it much easier to match the C# objects with the database tables. It saves time and prevents syntax errors from writing SQL manually.

2.2.2 RESTful API and Cryptographic Security

The client app communicates with the database only via a RESTful API, which I implemented using the AuthController. I used Dependency Injection to ensure the controller can access the AppDbContext whenever needed. I set up several HTTP POST endpoints like register, login, and syncChips to manage user account operations. For example, when someone registers, the code automatically gives them 5000 chips, so every new player starts with the same amount.

To ensure account security, user passwords are not stored in plain text. The system implements the SHA256 algorithm to hash passwords before storing them in the database.

Listing 2: Implementing SHA256 one-way hashing for secure password storage

```
private string HashPassword(string password)
{
    using (var sha256 = SHA256.Create())
    {
        var bytes = sha256.ComputeHash(Encoding.UTF8.GetBytes(password));
        return Convert.ToBase64String(bytes);
    }
}
```

During registration and authentication, passwords are processed through the HashPassword() method. Hashing the passwords into Base64 strings decreases the impact of potential database breaches.

I also built a way for users to get back into their accounts if they forget their passwords. I set up three steps for this: get-questions, verify-answers, and reset-password. If they answer their security questions correctly, the server lets them pick a new password. I then hash that new password and save it over the old one. Lastly, I used a syncChips part to keep track of the players' money. After a game ends, whether they played online or alone, this part securely updates the database to reflect how many chips they have left so they don't lose their progress.

2.3 Core Game Logic Implementation

The core game engine is responsible for enforcing the poker rules and controlling the game flow.

2.3.1 Hand Evaluation and Combinatorics Algorithm

Writing a poker evaluator from scratch is difficult and prone to errors, so I looked at open-source code to learn how other developers solved this problem. The core algorithmic logic was inspired by an open-source poker evaluator (Paz, 2021). I analysed their code to understand how it works, and then wrote a simpler version of it in my own HandEvaluator class.

To figure out the best hand, the code first puts the player's two cards and the five community cards together into a single pool of seven. Then, I used a recursive method to pick out every possible five-card combination from those seven cards. Combinatorially, this provides exactly 21 different five-card hands to check.

For each of those 21 hands, I wrote a method called Evaluate5Cards to see what kind of poker hand it is. I used LINQ (Language-Integrated Query) to make the code cleaner—it helps sort the cards from high to low and group them by rank. By looking at how big these groups are, the system can easily find things like pairs, three-of-a-kind, or four-of-a-kind. At the same time, the code checks for a flush by seeing if all the suits are the same, and it looks for a

straight by checking the gap between the highest and lowest cards. I also added a special "if" statement to handle the A-2-3-4-5 straight, since the Ace counts as a "1" in that case.

Finally, the code saves the result in a HandResult object, which stores the hand type and the "kicker" cards used for breaking ties. Because I made the HandResult class implement the IComparable interface, the system can automatically compare all 21 results, determine which is the strongest, and handle ties using the kickers.

To get all the possible five-card hands from the seven cards, I wrote a recursive function instead of using many complicated loops.

Listing 3: Recursive combination generation in `HandEvaluator.cs`

```
private static List<List<Card>> GetCombinations(List<Card> list, int length)
{
    if (length == 1) return list.Select(t => new List<Card> { t }).ToList();
    if (list.Count == length) return new List<List<Card>> { new List<Card>(list) };
    if (list.Count < length) return new List<List<Card>>();
    var result = new List<List<Card>>();
    var head = list[0];
    var tail = list.Skip(1).ToList();
    foreach (var combo in GetCombinations(tail, length - 1))
    {
        combo.Insert(0, head);
        result.Add(combo);
    }
    result.AddRange(GetCombinations(tail, length));
    return result;
}
```

I used a recursive way to break down the task of making card combinations into smaller pieces. The code picks the first card and then looks for more combinations in the rest of the cards. This aligns with the C (7,5) combinatorial logic and improves code maintainability.

Another challenge was figuring out who wins when players have the same hand type. I used the IComparable interface in my HandResult class. This lets the system compare hands and break ties automatically, which keeps the code clear to see and more efficient for debugging.

Listing 4: Tie-breaking logic utilising `IComparable` in `HandEvaluator.cs`

```
public class HandResult : IComparable<HandResult>
{
    public HandType Type { get; set; }
    public List<Card> Best5Cards { get; set; } = new List<Card>();
    public List<Rank> Kickers { get; set; } = new List<Rank>();
    public int CompareTo(HandResult? other)
    {
        if (other == null) return 1;
        if (this.Type != other.Type)
            return this.Type.CompareTo(other.Type);
        for (int i = 0; i < this.Kickers.Count; i++)
        {
            if (this.Kickers[i] != other.Kickers[i])
                return this.Kickers[i].CompareTo(other.Kickers[i]);
        }return 0;
    }
}
```

This implementation places the comparison logic directly inside the `HandResult` class. The method first checks the primary hand type. If two players have the same hand type, it uses a simple loop to check the sorted list of kickers to decide the winner. By using the `IComparable` interface, the main game controller does not need to process complex tie-breaking rules. This approach keeps the core game logic clean and makes the overall system much easier to read and maintain.

2.3.2 Computer Logic

To build the computer's decision logic, I used some ideas from an open-source project by Lilov (Lilov, 2021) and combined them with my own rules.

First, I grouped the hand strengths into different levels, similar to Lilov's approach. If the computer has a strong hand, such as two pair or better, it will always choose to call or raise without doing extra calculations.

Second, I made a risk-assessment feature for the computer player. I adapted the concept of "betting pressure" from open-source code and changed it. My system looks at the amountToCall and compares it to how many chips the computer has left. If the opponent bets more than one-third of the computer's total chips, the code uses a random number generator. This drops the chance of calling the bet to 50%. This change stops the bot from losing too many chips on bad cards when the other player is betting big.

I also added a bluffing feature inspired by Lilov's work. I set this to a 10% chance. This means that even if the computer has a very weak hand, it still has a 10% chance to call the bet anyway. Using this tactic makes the computer harder to predict, just like a real person. The specific implementation of this decision logic is demonstrated in the code below:

Listing 5: Pseudo-random betting logic in `ComputerLogic.cs`

```
public static bool DecideCallOrFold(Player computer, List<Card> communityCards,
int amountToCall)
{
    if (amountToCall >= computer.Chips) amountToCall = computer.Chips;
    HandResult current = HandEvaluator.GetBestHand(computer.HoleCards,
communityCards);
    Random rnd = new Random();
    int randomAction = rnd.Next(1, 101);
    // Always call with strong hands (Two Pair or better)
    if (current.Type >= HandType.TwoPair) return true;
    //  Marginal hands with risk assessment
    if (current.Type == HandType.OnePair || computer.HoleCards.Max(c =>
(int)c.Rank) >= (int)Rank.Jack)
    {
        // High betting pressure: 50% chance to call
        if (amountToCall > computer.Chips / 3)
        {
            return randomAction <= 50;
        }
        return true;
    }
    // Weak hands with a 10% pseudo-random bluffing chance
    return randomAction <= 10;
}
```

By combining these three levels of logic, the computer player acts more like a real person, making the offline practice mode more useful and challenging.

2.3.3 Original Card Swap Mechanism

Traditional Texas Hold'em relies on the initial card distribution and on psychological warfare through betting actions such as raising and calling. However, players have very limited direct control over their actual hand composition. To increase strategic complexity and player agency, I introduced an original "River Swap" phase. This mechanic allows players to exchange one of their hole cards after the river card is dealt, adding a new layer of decision-making that can turn a weak hand into a winning combination.

Human Player Implementation and Workflow

I made the human player's swap mechanic interactive and safe for the game state. This phase starts only after all five community cards are on the table, but before any players show their hands. At this point, the game moves to the River Swap stage, and a message pops up on the screen instructing the player on what to do.

The player can click on one of their own hole cards to pick it. Then they have two choices: they can click "Swap Selected" to get a new card from the deck, or click "Keep Hand" if they want to stay with their original cards. Once they make this final choice, the game goes straight to the Showdown. All players flip their cards over, and the system finds the winner.

Listing 6: UI event handler for the card swap feature in `OfflineGameWindow.xaml.cs`

```
private void DoSwapBtn_Click(object sender, RoutedEventArgs e)
{
    if (_selectedSwapIndex == -1)
    {
        MessageBox.Show("Please select a card to swap first!");
        return;
    }
    Card dropCard = _humanPlayer.HoleCards[_selectedSwapIndex];
    Card keepCard = _humanPlayer.HoleCards[1 - _selectedSwapIndex];

    _gameEngine.Deck.InsertCardRandomly(dropCard);
    Card newCard = _gameEngine.Deck.DealCard()!;

    _humanPlayer.HoleCards.Clear();
    if (_selectedSwapIndex == 0)
    {
        _humanPlayer.HoleCards.Add(newCard);
        _humanPlayer.HoleCards.Add(keepCard);
    }
    else
    {
        _humanPlayer.HoleCards.Add(keepCard);
        _humanPlayer.HoleCards.Add(newCard);
    }
    DisplayCardsOnPanel(_humanPlayer.HoleCards, PlayerHoleCardsPanel, true);
    CenterBroadcastToast.Visibility = Visibility.Collapsed; ExecuteShowdown();
}
```

As illustrated in the code snippet, the "Swap Selected" execution relies on precise index management. When the button is clicked, the code finds the card the player wants to drop, and then finds the kept card by using 1 - _selectedSwapIndex. I decided to call InsertCardRandomly(dropCard) before giving the player a new card. This is important because it puts the old card back into the deck, which keeps the game fair and the math correct. If the player clicks "Keep Hand," the code just skips the deck part. No matter what the player chooses, the system makes sure the hand is finished before it runs the ExecuteShowdown method. This final part of the code shows everyone's cards, compares the hands, and gives the chips to the winner.

Computer Player Decision Logic

Because I added the River Swap feature, I also had to update the computer player's code so the bots know how to use this new rule. The computer's swap logic is based on checking how weak its current hand is. A basic random swap would often destroy already winning hands. Therefore, the bot checks its current hand result first. It will only choose to swap a card if the hand is very weak.

Listing 7: Computer River Swap decision logic in `ComputerLogic.cs`

```
public static void DecideRiverSwap(Player computer, List<Card> communityCards,
Deck deck)
{
    HandResult currentBest = HandEvaluator.GetBestHand(computer.HoleCards,
communityCards);
    if (currentBest.Type == HandType.HighCard ||
       (currentBest.Type == HandType.OnePair && currentBest.Kickers[0] <
Rank.Ten))
    {
        Card cardToKeep;
        Card cardToDrop;
        if (computer.HoleCards[0].Rank >= computer.HoleCards[1].Rank)
        {
            cardToKeep = computer.HoleCards[0];
            cardToDrop = computer.HoleCards[1];
        }
        else
        {
            cardToKeep = computer.HoleCards[1];
            cardToDrop = computer.HoleCards[0];
        }
        deck.InsertCardRandomly(cardToDrop);
        Card? newCard = deck.DealCard();
        if (newCard != null)
        {
            computer.HoleCards.Clear();
            computer.HoleCards.Add(cardToKeep);
            computer.HoleCards.Add(newCard);
        }
    }
}
```

Firstly, it checks the `HandResult`. The bot is programmed to execute a swap only if it holds a completely unformed hand (High Card) or a highly vulnerable hand. It compares the numerical rank of the two-hole cards, specifically targeting the lower-value card to drop (`cardToDrop`) while retaining the higher-value card (`cardToKeep`). This gives the bot the best chance of getting a high pair with the new card, and increases the difficulty of the offline game mode.

In summary, the Card Swap mechanism adds a new strategic layer to the standard rules. It gives players more control over their final hand and increases the decision-making complexity for both human and computer players.

2.4 Game State Machine and Action Handling

Texas Hold'em follows a strict set of rules. To make the server enforce these rules and stop illegal moves, I built a Finite State Machine (FSM). This FSM ensures the game progresses logically from one phase to the next and accepts player moves only when they are allowed in that state.

The heart of this FSM is a _gameState variable on the server. A normal round follows a straight path through these steps: Waiting, PreFlop, Flop, Turn, River, RiverSwap, and Showdown.

To move between these states, I wrote an AdvancePhase() method. This method acts like the engine for the state machine. It automatically starts the next part of the game once every active player has finished betting and matched the current bet.

Listing 8: State transition logic managing the dealing of community cards in GameHub.cs

```
private async Task AdvancePhase()
{
    await Task.Delay(1000);
    // Reset round-specific betting variables
    foreach (var p in _players) { p.CurrentBet = 0; p.HasActedThisRound =
false; }
    _currentMaxBet = 0; await SyncTableState();

    // Finite State Machine transitions
    if (_gameState == "River")
    {
        _gameState = "RiverSwap";
        await Clients.All.SendAsync("SystemMessage", "Final River Swap!");
        await SyncTableState(); _currentTurnIndex = -1; await NextTurn();
return;
    }
    if (_gameState == "RiverSwap") { await EvaluateShowdown(); return; }

    // Deal community cards based on the new state
    if (_gameState == "PreFlop") { _gameState = "Flop"; for (int i = 0; i < 3;
i++) { _communityCards.Add(_deck.DealCard()); } }
    else if (_gameState == "Flop") { _gameState = "Turn";
_communityCards.Add(_deck.DealCard()); }
    else if (_gameState == "Turn") { _gameState = "River";
_communityCards.Add(_deck.DealCard()); }
    var allCardImages = _communityCards.Select(c => c.ImageFileName).ToList();
    await Clients.All.SendAsync("ReceiveCommunityCards", allCardImages);

    var activePlayers = _players.Where(p => !p.HasFolded).ToList();
    if (activePlayers.Count(p => p.CorePlayer.Chips > 0) <= 1) { await
AdvancePhase(); }
    else { _currentTurnIndex = -1; await NextTurn(); }
}
```

As demonstrated in Listing 8, the system evaluates the current _gameState and deals the community cards accordingly. For instance, transitioning from PreFlop to Flop triggers the dealing of three community cards, whereas transitioning to Turn or River deals exactly one. It

also resets temporary variables, such as CurrentBet, to enforce the betting rules for the upcoming round.

All intra-phase player actions are processed within the FSM. The server calculates the amount of decreased chips and makes sure the game only moves to the next step when all active players have matched the current bet.

2.5 Real-time Network Communication

To make the multiplayer mode work, the game needed a fast network. Normal HTTP requests are not good enough because they are too slow for real-time multiplayer games. To resolve this, I implemented an Authoritative Server architecture utilising Microsoft SignalR (Microsoft,2023), enabling persistent, full-duplex communication between the server and all connected players.

2.5.1 SignalR Configuration and WebSocket Protocol

On the server side, the GameHub class inherits from SignalR's Hub base class to handle real-time messages. On the client side, I integrated Microsoft.AspNetCore.SignalR.Client NuGet package. This industry-standard library automatically manages the underlying connection protocols. It attempts to establish a high-performance WebSocket connection by default, but provides automatic fallbacks if the user's firewall blocks WebSockets.

Listing 9: Establishing the SignalR connection in the client application from `OnlineGameWindow.xaml.cs`

```
private async void ConnectToServer()
{
    // Build the persistent connection to the server's GameHub
    _connection = new HubConnectionBuilder()
        .WithUrl("https://localhost:7140/gamehub")
        .Build();
    // Register event listener for incoming broadcast messages
    _connection.On<string, string>("ReceiveMessage",
        (user, message) => AddChatMessage($"{user}: {message}"));
    // Register event listener for system state updates
    _connection.On<string>("SystemMessage", (sysMsg) =>
    {
        // Internal localisation mapping logic omitted for brevity
        string translated = sysMsg;
        AddChatMessage($"[System] {translated}");
    });
    await _connection.StartAsync();
}
```

As shown in Listing 9, the client initiates a connection using the HubConnectionBuilder and connects to the server at the specified URL. The most critical part of this implementation is the use of on handlers. These event listeners operate asynchronously in the background and wait for specific broadcast events such as `ReceiveMessage` and `SystemMessage` from the server. Once the server pushes a payload, these handlers intercept the data and process internal formatting, such as UI localisation mapping. This ensures that real-time updates are rendered to the client's chat interface. Finally, the `StartAsync` method opens the persistent WebSocket tunnel.

2.5.2 Authoritative Server Implementation

In multiplayer game development, trusting the client is a significant security flaw. To prevent cheating, such as unauthorised card modifications or false chip injections, I implemented an Authoritative Server pattern within the GameHub.cs. The server retains absolute control over the _deck, _communityCards, and _pot static variables.

Listing 10: Server-side logic handling player connection and verification from GameHub.cs

```
public async Task JoinGame(string playerName)
{
    // Prevent duplicate logins
    var existing = _players.FirstOrDefault(p => p.CorePlayer.Name ==
playerName);
    if (existing != null) _players.Remove(existing);

    // Enforce maximum table capacity
    if (_players.Count >= 6) { await Clients.Caller.SendAsync("SystemMessage",
"Room is full!"); return; }

    // Verify account balance directly from the database, not from client input
    var dbUser = _dbContext.Users.FirstOrDefault(u => u.Username ==
playerName);
    int realChips = dbUser != null ? dbUser.Chips : 5000;

    if (realChips <= 0) { await Clients.Caller.SendAsync("KickedOut",
"Insufficient chips!"); return; }
    // Register player securely and broadcast update
    _players.Add(new ServerPlayer { CorePlayer = new Player(playerName,
realChips), ConnectionId = Context.ConnectionId });
    await Clients.All.SendAsync("SystemMessage", $"🪑 Player【{playerName}】is
sitting on the table");
    await SyncTableState();
}
```

When a user attempts to join a table via the JoinGame method, the server does not simply trust the chip balance the client sends. Instead, it queries the AppDbContext to retrieve the verified chip amount directly from the MySQL database. Also, it enforces the six-player limit. If the validation passes, the server associates the specific ConnectionId with a new ServerPlayer object, ensuring that any subsequent actions are mapped to the correct user.

2.5.3 Real-Time State Synchronisation Flow

The core mechanism for keeping all clients synchronised is the SyncTableState method within the GameHub. When a player performs an action such as clicking the Call button, the client invokes the PlayerAction method on the server. The server then calculates the new pot size and deducts the chips from it. Instead of sending the result only to that specific player, the server triggers Clients.All.SendAsync with the UpdateTable event.

This method packages the current state of all players, including their names, seat positions, and betting status, along with the pot size, the current max bet, and the overall game state

into an anonymous object. This data is pushed over the WebSocket to all connected clients at same time. By using this broadcasting approach, the system ensures that all clients render the same table state within milliseconds. This solves the problem of real-time updates in a multiplayer environment.

3. Testing and Evaluation

The testing phase is divided into two main parts: Black-Box Functional Testing to verify the core game mechanics and network stability, and User Acceptance Testing to gather real player feedback and improve the user interface.

3.1 Black-Box Functional Testing

To check the system's reliability, I used Black-Box Functional Testing. Instead of testing every simple UI button, I focused on the most complex parts: the core game logic, the swap mechanic, and network errors.

Test Case 1: Core Game Logic and Tie-breaking
Objective: To verify the accuracy of the 7-choose-5 combinatorial algorithm and ensure the HandEvaluator correctly resolves complex hand rankings.

Action: Using the offline training mode to artificially manipulate a showdown. Player 1 is dealt a Flush (five hearts), while computer player is dealt a Full House (three 8s and two 5s).

Expected Result: The system must process all 21 possible combinations, correctly identify that a Full House is mathematically stronger than a Flush according to standard Texas Hold'em rules, and automatically award the accumulated pot to Player 2.

Actual Result: Passed. The system correctly recognised the Full House, highlighted Player 2's avatar to indicate the win, and updated the chip balance.



Figure 10: The system resolves a showdown between a Full House and a Flush.

Test Case 2: Original Card Swap Data Integrity
Objective: To ensure that the introduced "River Swap" feature does not corrupt the deck's underlying data structure or violate the strict two-card hole card rule.

Action: During an active game session, progress to the River Swap phase. Select one of the existing hole cards and execute the "Swap Selected" command.

Expected Result: The system must accurately remove exactly one card from the player's hand array and securely insert it back into the Deck array. It must then randomly deal exactly one new card. The player's final hand size must remain at 2, preventing any array out-of-bounds exceptions, and the game must proceed to the Showdown phase.

Actual Result: Passed. The user interface dynamically refreshed to display the newly dealt card. The hand size remained perfectly synchronised at two cards, and the system proceeded to calculate the final hand evaluation without any data corruption.

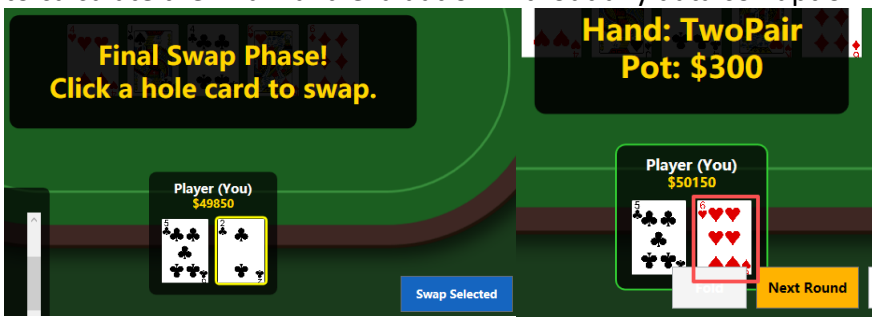


Figure 11: State transition before (left) and after (right) executing the custom River Swap mechanic. The highlighted card is successfully replaced while maintaining a valid two-card hand size, demonstrating data integrity.

Test Case 3: SignalR Network Stability and Disconnection Handling

Objective: To evaluate the Authoritative Server's edge-case handling capabilities when a client drops the WebSocket connection.

Action: Launch two client test02 and test03 to join the same online table. During an active betting round, terminate test03's application window to simulate a network dropout.

Expected Result: The GameHub server must not crash. It must catch the OnDisconnectedAsync event and instantly broadcast a system notification to the surviving client (test02) to maintain state synchronisation.

Actual Result: Passed. The server handled the abrupt disconnection. The remaining client instantly received the system broadcast [System] Player 【test03】 left the table. within the localised Game Log, proving stable network state management.

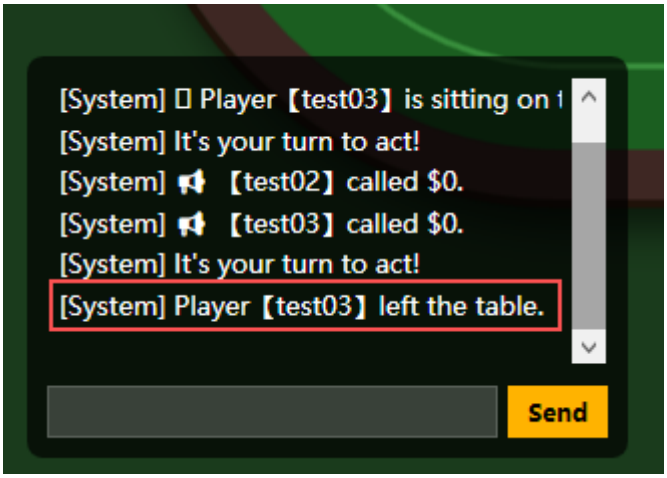


Figure 12: The Game Log captures an abrupt client disconnection and broadcasts the event to the surviving player.

Test Case 4: Database Integrity and Authentication Security

Objective: To validate the backend database layer's ability to handle constraint violations and ensure user credentials are securely processed.
Action: Use the registration interface to attempt to create a new account with the username test02, which already exists in the database.

Expected Result: The backend Authentication API must catch the SQL constraint violation, reject the request safely without crashing the server, and return an error to the client. Database inspection must confirm that all passwords are securely hashed rather than stored in plain text.

Actual Result: Passed. The system successfully blocked the duplicate registration and displayed a "Username already exists" warning. Direct database inspection confirmed that user passwords are encrypted using cryptographic hashing, ensuring data security.

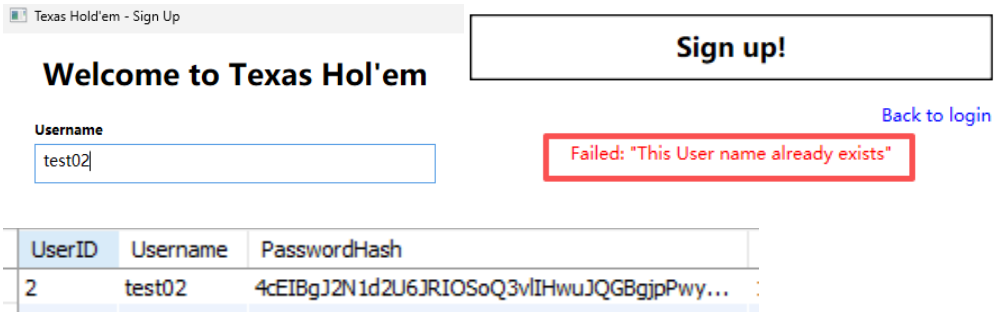


Figure 13: End-to-end validation of database integrity and security. The system successfully rejects duplicate user registrations while maintaining cryptographic hashing for user credentials in the database.

3.2 User Acceptance Testing

To test the game with real users, I ran a User Acceptance Testing (UAT) session with five participants, including my classmates and friends. They played the game and filled out a short survey on Microsoft Forms (Microsoft, 2024) to rate the usability.

The questionnaire scores were good, but their written feedback helped me find several bugs that I didn't see during my own testing. The following subsections detail the core issues identified during the UAT phase and the subsequent iterative fixes implemented in the system.

3.2.1 Domain Terminology Inaccuracy: The "Call $0" Issue

Problem Description:

The open-ended feedback in the survey helped me find a problem with the poker terms in the Game Log. As captured directly from the survey submission, when a player chose to match a current bet of zero, the log incorrectly outputted the string "[Player] called $0." While mathematically correct within the backend state machine, the accurate domain-specific terminology for this action in poker is "Check".

1	anonymous	When no one raises, the log says 'called $0'. In real poker, this should just be 'Check'.

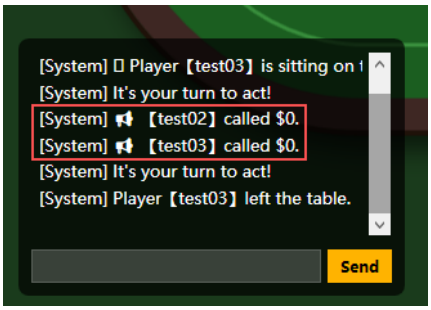


Figure 14: The system game log incorrectly displays "called $0" instead of the standard poker terminology "Check" prior to the fix.

Root Cause Analysis:

Upon reviewing the server logs and backend logic, the root cause was identified in the PlayerAction method within the GameHub.cs class. The server processed all matching bets under the "Call" action logic, deducting the calculated amountToCall from the player's balance. However, it lacked a conditional branch to verify if the calculated difference was exactly zero before broadcasting the system message.

Resolution and Code Implementation:

To use the correct poker terms and improve the user experience, I added an if-else statement in the backend. The backend was modified to distinguish between a true call and a check, broadcasting the precise terminology. Also, the client-side OnlineGameWindow.xaml.cs localization dictionary was updated to map the Chinese action correctly to the English term "checked".

The core logic refactoring is demonstrated in Listing 11:

```
if (amountToCall == 0)
{
    await Clients.All.SendAsync("SystemMessage", $"📢 【{player.CorePlayer.Name}】check.");
}
else
{
    await Clients.All.SendAsync("SystemMessage", $"📢 【{player.CorePlayer.Name}】call ${amountToCall}。");
}
```

Actual Result:

Following the hotfix, the system reflects the player's intent. When no active bet is present, the game log displays "[Player] check", resolving the UX discrepancy and passing the domain accuracy test.

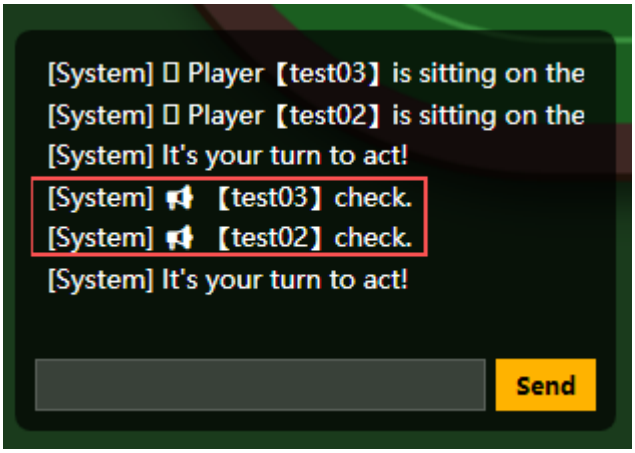


Figure 15: The resolved game log accurately displays the domain-specific "checked" terminology after the iterative update.

Evaluation:

This improvement ensures the software strictly follows professional poker rules. Although the original "Call $0" was mathematically correct, it was confusing for players. By using the correct "Check" terminology, the user interface provides a more authentic experience. This change enhances the game's professionalism and meets the expectations of experienced poker players.

3.2.2 Edge Case Resolution: Real-time State Deadlock in 1v1 Scenarios

During the User Acceptance Testing, a logical error was found in the 1v1 mode. If one player closed the application, the server would remove them from the list, but the game state did not change. This left the remaining player stuck in the game indefinitely because the server was still waiting for the disconnected player to act.

2	anonymous	Whilst I was testing the online mode, I noticed that if only two players are on the field and one of them leaves the game, the game continues to wait for the player who left rather than ending the match.

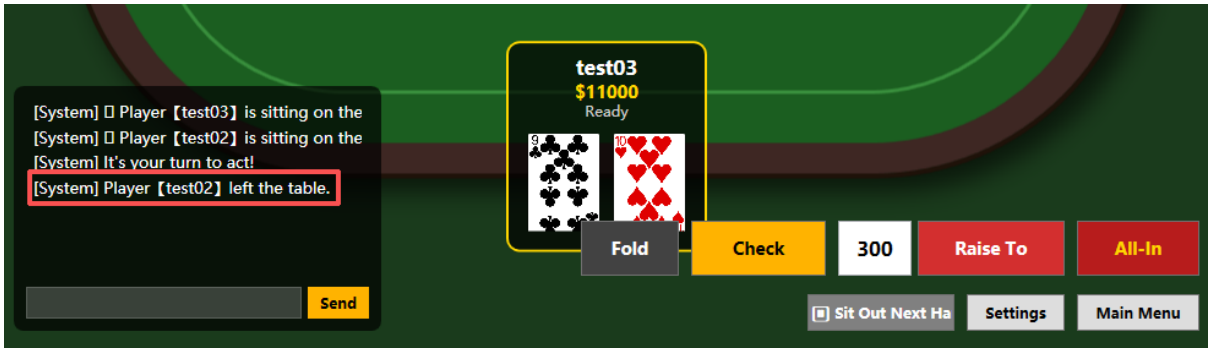


Figure 16: While there's only one player on the table, the game didn't stop

Root Cause Analysis:

The issue happened because the original OnDisconnectedAsync method only updated the player list. It did not check whether the number of remaining players was enough to continue the game. In a 1v1 match, once a player leaves, the game must end immediately, but the server-side state machine stays in the "Active" stage.

Solution and Implementation

To fix this, the disconnection logic in GameHub.cs was refactored to include an immediate player count check:

If only one active player remains after a disconnection, the system now calls the DeclareWinner method to award the pot to that player.

If no players are left, the system calls ResetForNextRound to clear all community cards and reset the game state to "Waiting".

Listing 12: Disconnected player method in GameHub.cs

```
public override async Task OnDisconnectedAsync(Exception? exception)
{
        // Check if the game is in progress and handle the disconnection accordingly
        if (_gameState != "Waiting")
        {
            // Get the list of active players who haven't folded yet
            var activePlayers = _players.Where(p => !p.HasFolded).ToList();

            if (activePlayers.Count <= 1)
            {
                // if only one player remains, declare them the winner by default
                if (activePlayers.Count == 1)
                {
                    await DeclareWinner(activePlayers[0], "Player disconnected");
                }
                // if no players remain, prepare for the next round without declaring a
winner
                else
                {
                    await ResetForNextRound();
                }
            }
            // if the disconnected player is the one whose turn it is, treat it as a fold
and move to the next turn
            else if (_currentTurnIndex >= 0 && _currentTurnIndex < _players.Count &&
_players[_currentTurnIndex] == player)
            {
                await PlayerAction("Fold 弃牌", 0);
            }
            else
            {
                await SyncTableState();
            }
        }
        else
        {
            await SyncTableState();
        }
}
```

Actual Result:

Testing showed that this fix successfully solved the problem. After the update, when a player disconnects in a 1v1 game, the surviving player receives the chips and the game table resets within one second. This ensures the game is more stable and prevents the server from getting stuck in an invalid state.



Figure 17: test02 gets all chips because test03 is disconnected from the game

Evaluation:
This fix successfully addressed a critical reliability issue in the multiplayer mode. By implementing an immediate settlement check, the system prevents the "soft-lock" state that previously occurred during disconnections. This ensures that the surviving player's time and chip balance are protected, which is essential for maintaining a fair and competitive online environment. It demonstrates that the server can handle network instability.

3.2.3 Session Security and Concurrent Login Resolution

During the UAT phase, qualitative feedback identified a critical session management vulnerability. It was discovered that a single user account could be logged into the system from two different client instances. This "concurrent login" flaw allowed the same identity to enter the game environment twice, which disrupted the server's state synchronisation and risked data corruption regarding the player's chip balance.

3	anonymous	When I attempted to log in to my account in the online mode, I noticed that logging in with the same account twice did not result in any restrictions or prompts. Both attempts allowed me to enter the online mode. However, this would cause a bug and prevent me from continuing the game.

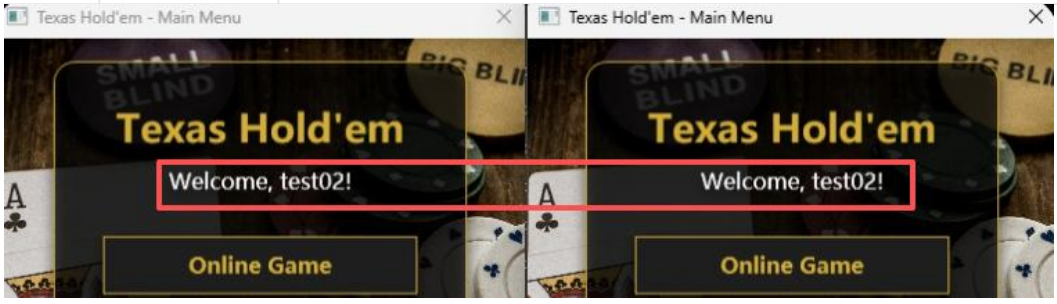


Figure 18: Same account login in different programs at the same time

Iterative Resolution and Lifecycle Challenges:
Fixing this login bug meant I had to rewrite the session management code on the server.

Phase 1: Implementing a Global Session Lock

To resolve the concurrent login issue, the first iteration focused on the server-side authentication. A thread-safe SessionManager was introduced to register active usernames.

Listing 13: Initial Backend Session Lock Logic

```
if (TexasHoldemServer.Hubs.SessionManager.IsUserLoggedIn(username))
{
    return BadRequest("This account has already been logged in elsewhere!");
}

TexasHoldemServer.Hubs.SessionManager.LoginUser(username);

[HttpPost("logout")]
public IActionResult Logout([FromQuery] string username)
{
    if (!string.IsNullOrWhiteSpace(username))
    {
        TexasHoldemServer.Hubs.SessionManager.LogoutUser(username);
    }
    return Ok(new { message = "Logged out" });
}
```

```
public static bool IsUserLoggedIn(string username)
{
    lock (_loggedInUsers) { return _loggedInUsers.Contains(username); }
}

public static void LoginUser(string username)
{
    lock (_loggedInUsers) { _loggedInUsers.Add(username); }
}

public static void LogoutUser(string username)
{
    lock (_loggedInUsers) { _loggedInUsers.Remove(username); }
}
```

Phase 2: The "Zombie Session" Anomaly

While Phase 1 addressed the concurrent login issue, it introduced a high-severity side effect. Because the server is now strictly locking the account, a new problem has emerged when users unexpectedly terminate the application.

Since the Windows OS process teardown happens immediately, it killed the client application before the asynchronous HTTP Logout request could be dispatched. Consequently, the server never received the signal to release the lock, creating a "Zombie Session." This left the user locked out of their account when they tried to log back in.

Phase 3: WPF Lifecycle Interception

To fix the concurrent login bug, the client must tell the server to log out before the window completely closes. To do this, I overrode the window closing event to stop the app from closing immediately until the logout request is sent.

As shown in Listing 14, I overrode the WPF window's OnClosing event. By setting e.Cancel = true causes the application to prevent the operating system from closing the window, allowing the main UI thread to keep control.

Listing 14: Client-side teardown interception guaranteeing account unlock upon unexpected exit.

```
private bool _isSafeToClose = false;

protected override async void OnClosing(System.ComponentModel.CancelEventArgs e)
{
    if (isNavigating || _isSafeToClose) { base.OnClosing(e); return; }

    // Phase 3 Core: Intercept and halt the OS-level termination process
    e.Cancel = true;

    MessageBoxResult result = MessageBox.Show("Are you sure you want to exit the game?",
                                              "Exit Confirmation", MessageBoxButton.YesNo);
    if (result == MessageBoxResult.Yes)
    {
        if (!string.IsNullOrEmpty(GameSession.CurrentUsername))
        {
            try
            {
                // Dispatch the unlock signal with SSL bypass and a strict timeout
                var handler = new HttpClientHandler
{ ServerCertificateCustomValidationCallback = (m, c, ch, er) => true };
                using (var client = new HttpClient(handler) { Timeout =
TimeSpan.FromSeconds(3) })
                {
                    await
client.PostAsync($"https://localhost:7140/api/auth/logout?username={GameSession.CurrentUsern
ame}", null);
                }
            }
            catch { /* Graceful degradation on network failure */ }
        }

        // Mark as safe and explicitly execute program termination
        _isSafeToClose = true;
        Application.Current.Shutdown();
    }
}
```

Actual Result:
The system now enforces a single active session per user identity, managing unpredictable application closures without leaving ghost records in the database.



Figure 19: Validation of the Concurrent Login Prevention mechanism, demonstrating the rejection of a secondary authentication attempt for an active session.

Evaluation:
Using backend session locking and catching the window closing event successfully fixed the concurrent login and 'Zombie Session' problems, keeping the players' chip data safe.

3.3 User Acceptance Testing Result Analysis

As shown in Figure 20, feedback from the five respondents was generally positive, especially in the "Usability" and "Gameplay" categories. Most users agreed that the interface is easy to navigate and the "River Swap" mechanic is interesting.

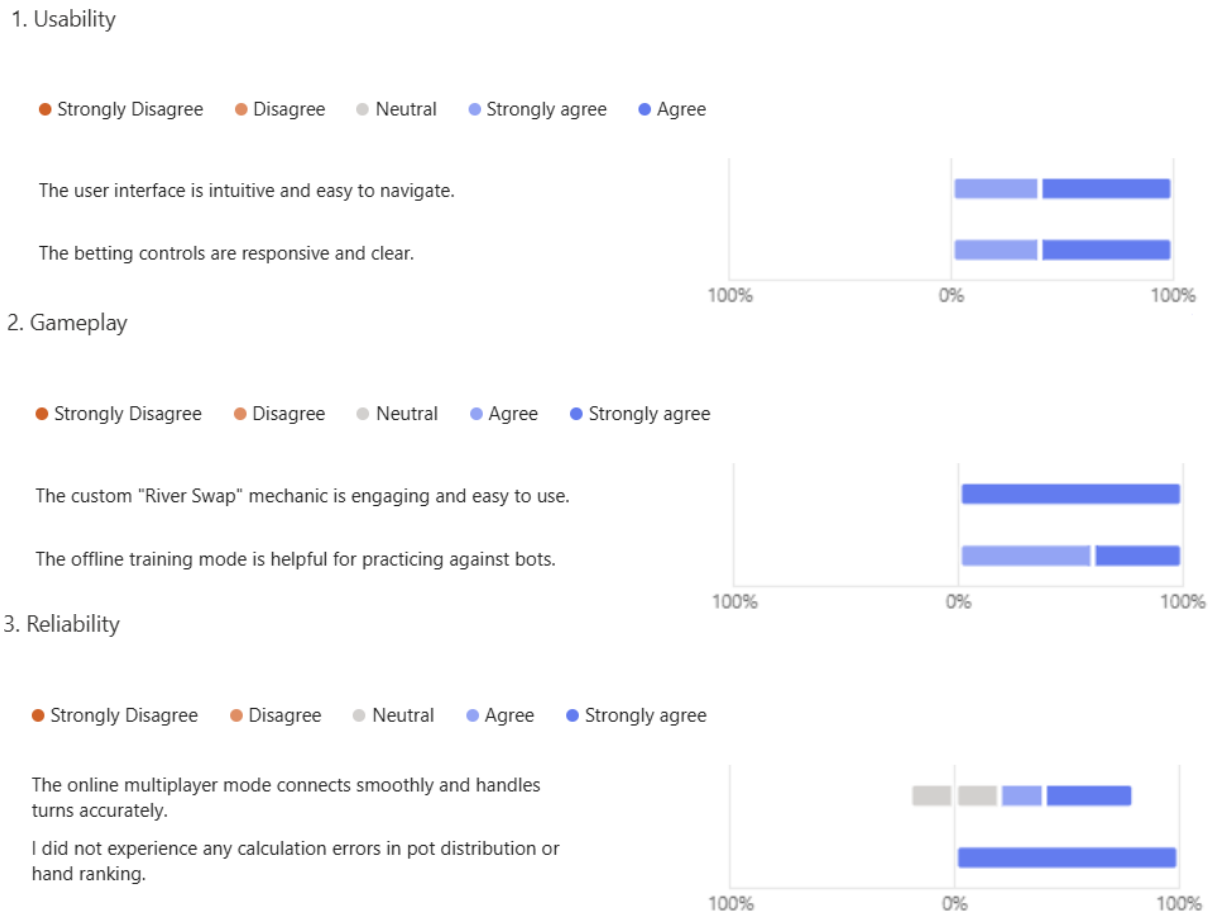


Figure 20: Result of the questionnaire of Texas Hold'em

The lower scores in the "Reliability" section directly correlate with the network and session management edge cases identified during testing

3.4 Summary Evaluation

In summary, the testing results and user feedback show that the game is stable and safe to play now. I have checked all the main technical parts, like the hand evaluation algorithm, the network connection, and the database, and they all work correctly. By fixing the bugs found in the UAT stage—especially the disconnect problem and the login security issue—the software now does everything I planned in Chapter 1. The Authoritative Server design is really helpful to stop players from cheating on their clients. At the same time, using SignalR makes sure the multiplayer connection is stable for everyone in the room.

4. Project Ethics

This project was conducted in strict accordance with the University of Liverpool's Policy on Research Ethics and the Code of Conduct of the Chartered Institute for IT. The research was classified as low risk because it focused on software implementation and relied on anonymous feedback for evaluation.

4.1 Ethical Compliance Categories

Data Category A: The project does not utilise any pre-existing datasets derived from humans or animals from open sources or public APIs. Although open-source code was studied to implement core algorithms, this material is treated as technical literature rather than a human-derived dataset.

Participant Category 2: Human participants were involved only in the software evaluation phase. In line with university guidance, a brief questionnaire was used to present the usability of the final product and the "River Swap" creative function.

4.2 Informed Consent

I invited five people to participate in User Acceptance Testing (UAT). Before we started, I gave each person an information sheet, and they signed a consent form. I made it very clear that they did not have to do the testing if they didn't want to, and they could stop and leave the activity at any time.

4.3 Use of Dummy Data

To protect everyone's privacy, I used dummy data during the system tests. I told the participants to use fake names like "test02" and "test03" when testing the sign-up and login. My application did not collect or save any real personal information, such as real names or private passwords.

4.4 Data Anonymity and Protection

For the feedback, I used an anonymous Microsoft Forms survey. This way, the results were mixed together, and I could not see who said what. All testing notes and signed consent forms are kept safely on a device protected by a password. Following the university rules, I will delete all the participant data forever after I graduate.

5. Conclusion & Future Work

5.1 Conclusion

This project completed all the main goals and requirements that I planned in Chapter 1. I built a real-time multiplayer Texas Hold'em game that works well. In conclusion, by using an Authoritative Server and SignalR, this project fixed common problems in online games such as players' disconnection without game termination and simultaneous account logins. It successfully connected the core game rules with real-time networks, creating a well-functioning multiplayer poker game. The server manages all the game rules and updates the chips through Entity Framework Core. This makes sure that the game environment is fair for all players.

The custom-built hand evaluation algorithm successfully resolved complex tie-breaking scenarios and correctly identified winning hands. Also, the successful implementation of the innovative "River Swap" mechanic—supported by both human UI logic and a dedicated computer decision model—added more strategy to the traditional rules, fulfilling the BCS requirement for innovation.

Finally, the testing part was highly valuable for refining the project. During the User Acceptance Testing, I found some serious bugs, such as problems when two people logged in at the same time and errors when one player disconnected. By fixing these edge cases, the software changed from a simple prototype into a stable and reliable system. To sum up, this project successfully delivered a functional multiplayer poker application, combining real-time network synchronization with a reliable evaluation algorithm.

5.2 Future work

5.2.1 Advanced AI via Reinforcement Learning

The current computer logic in the offline training mode relies on static heuristics and pseudo-random probabilities. For incomplete information games like Texas Hold'em, future iterations could integrate state-of-the-art machine learning algorithms, specifically Counterfactual Regret Minimisation. As demonstrated by Pluribus AI, which achieved superhuman performance in six-player no-limit poker (Brown and Sandholm, 2019), adapting advanced self-play algorithms would enable offline bots to dynamically adjust their betting strategies in response to opponents' behaviour. Using these algorithms would make the computer players much smarter and harder to beat in the offline training mode.

5.2.2 Scalability via Redis Backplane

During my research, I found that SignalR is very good for a small number of players. However, if the game needs to support thousands of players simultaneously, a single server will not be enough. In the future, I plan to use Redis as a backplane to help the system scale up. This tool would connect multiple servers and ensure that all game messages are sent correctly to every player, even if they are connected to different servers (Microsoft, 2025).

5.2.3 Competitive Social and Ranking Ecosystem

To enhance long-term player engagement, the application could be expanded with a comprehensive social framework. This would include a Friends System for private matches and an Elo-based competitive ranking system, and providing detailed player profiles with advanced statistics, such as Win Rate and Voluntarily Put Money in Pot, would add

competitive depth for the game.

6. BCS Project Criteria and Self-Reflection

During the development of this Texas Hold'em game, I ensured my work met the standards of the Chartered Institute for IT (BCS). Looking back at what I have done, I can evaluate my project and my skills based on these six criteria:

6.1 An ability to apply practical and analytical skills gained during the degree programme

I applied my foundational Object-Oriented Programming and algorithm design skills to build the complex hand-evaluation engine. I also learned how to use Microsoft SignalR to build the multiplayer network, which was a completely new technical skill for me.

6.2 Innovation and/or creativity

This criterion was met by designing and implementing the unique "River Swap" mechanic. Instead of a standard poker game, I added a creative strategic layer to the traditional rules, which required me to update both the user interface and the backend code.

6.3 Synthesis of information, ideas and practices to provide a quality solution together with an evaluation of that solution

I combined different skills I learned in my degree, such as network programming, database management, and UI design, to build a working software project. The quality of this solution was then evaluated and refined through Black-box testing and User Acceptance Testing.

6.4 That your project meets a real need in a wider context

The project demonstrates a practical solution to client-side cheating in multiplayer card games by implementing an Authoritative Server architecture.

6.5 An ability to self-manage a significant piece of work

Overall, I managed the workload of this project by planning the tasks stage by stage in Gantt chart. Because of my prior experience with C# and WPF, I finished the initial UI development and basic logic on time. However, I significantly underestimated the time required to learn and implement Microsoft SignalR. The complexity of real-time network synchronisation caused delays that pushed back the schedule's later stages. Therefore, the project schedule was adjusted to prioritise core backend stability over extended UI refinements.

6.6 Critical self-evaluation of the process.

Throughout the development process, I identified several areas for improvement in my technical approach and workflow:

System Integration Complexity: While I was familiar with C#, WPF, and SQL databases individually, I found that integrating them with a real-time protocol added unexpected complexity. My main technical hurdle was maintaining state consistency across the client UI, server logic, and database.

User Acceptance Testing: I initially underestimated the time required for the testing phase. When I conducted the UAT, participants interacted with the system in unpredictable ways, triggering edge cases that my standard functional testing did not catch. This experience

showed me the importance of allocating sufficient time for real-user testing to ensure system stability.

Workflow and Architecture: A major flaw in my workflow was not setting up a proper version control system from the start. I relied on manual code rollbacks during debugging, which made my development process inefficient. Furthermore, I coupled the core game logic too tightly with the network communication code. Overall, I should have developed and tested the offline game engine before integrating the multiplayer features to modularize the architecture and simplify debugging. This would make testing and evaluation stage of online game much easier.

Reference

Brown, N. and Sandholm, T. (2019) 'Superhuman AI for multiplayer poker', Science, 365(6456), pp. 885-890. Available at: https://www.science.org/doi/10.1126/science.aay2400 (Accessed: 7 May 2026).

HD Poker (2026) HD Poker: The High Stakes Social Poker Game. Available at: https://hd.poker/ (Accessed: 1 April 2026).

Kenney (2026) Interface Sounds. [online] Available at: https://kenney.nl/assets/interface-sounds (Accessed: 20 April 2026).

Lilov, D. (2021) Team-Breadfruit: Bot.cs. [online] GitHub. Available at: https://github.com/DimitarLilov/Team-Breadfruit/blob/b40a1f858f9a717703f7bb23a99fbcca0a64c3ae/Poker/Models/Players/Bot/Bot.cs (Accessed: 1 May 2026).

Microsoft (2023) Overview of ASP.NET Core SignalR. [online] Available at: https://learn.microsoft.com/en-us/aspnet/core/signalr/introduction (Accessed: 25 April 2026).

Microsoft (2024) Microsoft Forms: Create an inclusive survey, quiz, or poll. [online] Available at: https://forms.microsoft.com/ (Accessed: 2 May 2026).

Microsoft (2025) Set up a Redis backplane for ASP.NET Core SignalR scale-out. [online] Microsoft Learn. Available at: https://learn.microsoft.com/en-us/aspnet/core/signalr/redis-backplane (Accessed: 7 May 2026).

Paz, D. (2021) Poker-Hand-Evaluator. [online] GitHub. Available at: https://github.com/danielpaz6/Poker-Hand-Evaluator (Accessed: 1 May 2026).

Pixabay (2026) Pixabay. [online] Available at: https://pixabay.com/ (Accessed: 20 April 2026).

Appendix A: Installation Guide and User Manual (README)

1. Project Overview

This application is built using a Client-Server architecture. The TexasHoldemServer (backend) manages all game logic and data persistence, while the TexasHoldemClient (frontend) provides the user interface. For the game to function, the server must be initialised and running before any clients attempt to connect.

2. System Prerequisites

To run the project, the following software must be installed on the host machine:

IDE: Visual Studio 2022
Framework: .NET 8.0 SDK
Database: MySQL Server (Version 8.0 or higher) and MySQL Workbench

3. Database Setup (Crucial First Step)

Since the system requires a local MySQL database for user authentication and chip persistence:

Open MySQL Workbench.

Execute the provided Database_Setup.sql script located in the project root directory. This will create the required schema and pre-populate the test accounts.

Open the TexasHoldemServer project in Visual Studio.

Update the appsettings.json file by replacing the Uid and Pwd values in the DefaultConnection string with your local MySQL root username and password.

4. Running the Application

To simulate a multiplayer match on a single machine:

Launch the Server: Set TexasHoldemServer as the Startup Project in Visual Studio and press F5. A console window will open, indicating the server is active. Do not close this window.

Launch the Client: Set TexasHoldemClient as the Startup Project and press F5 to start the first player's interface.

Multiplayer Simulation: Navigate to the client build directory (\TexasHolem\TexasHolem\bin\Debug\net8.0-windows\) and double-click the TexasHoldemClient.exe file to open additional game windows.

5. Test Credentials and Features

The system includes pre-configured accounts for immediate evaluation:

User 1: Username: test02 | Password: test02
User 2: Username: test03 | Password: test03

The River Swap Mechanic:
After the final community card is dealt, the game enters the custom "River Swap" phase. Click on one of your hole cards to select it, then click "Swap Selected" to receive a new card from the deck, or click "Keep Hand" to proceed to the final showdown.